# Intermediate DevOps
# Day 3: Containers, Kubernetes and continuous delivery

United Airlines · Pluralsight instructor-led training

Kangeyan Passoubady
9-11 November 2026



KavinSchool

DISCUSSION

# Recall: Prepare a deployment

Without opening your notes, explain yesterday's prerequisites:

1. How do reviewed infrastructure changes reach the cluster?
2. What protects shared state from concurrent writers?
3. How would you check that a deployment identity has the intended permissions?

Think privately for 60 seconds. Share a sentence only if you wish, then correct it using the next slide.

ANSWER

# Day 2 recap

- Infrastructure as code describes the desired state in version-controlled files.
- Terraform compares the configuration with its state and plans the changes.
- The `platform` repository runs a plan on every pull request and applies on merge.
- Terraform state is stored in a Kubernetes Secret. A Lease stops two runs from applying at once.
- Credentials get the least privilege and the shortest lifetime that still work. A pipeline never prints them.

Your cluster now has a dev and a prod namespace, each with a quota and a Postgres database.

# Day 3 agenda

## MORNING SESSIONS — AM

| | | |
|---|---|---|
| 1 | RECAP | Day 2 recap |
| 2 | LECTURE | Containers |
| 3 | LECTURE | Docker Compose, Amazon ECS and Fargate |
| 4 | LAB 07 | Build, scan and describe a container |
| 5 | LECTURE | Kubernetes |

## AFTERNOON SESSIONS — PM

| | | |
|---|---|---|
| 6 | LAB 08 | Kubernetes deployments, rollouts and rollbacks |
| 7 | LECTURE | Continuous delivery |
| 8 | LAB 09 | End-to-end delivery pipeline |
| 9 | WRAP-UP | Course summary and feedback |

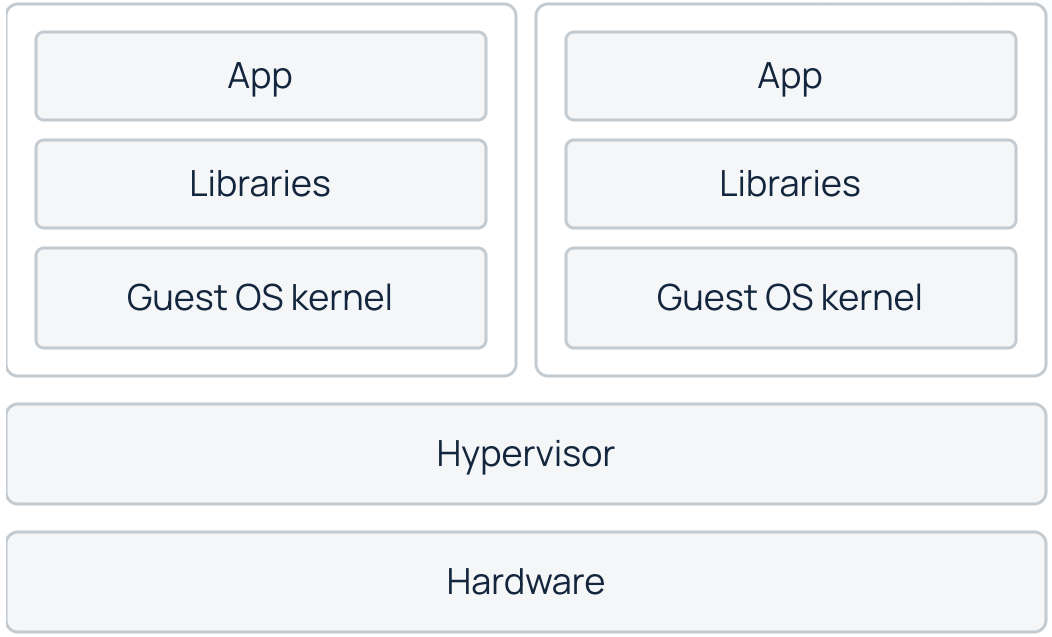
AWS: Amazon Web Services. ECS: Elastic Container Service.

# Containers and Docker

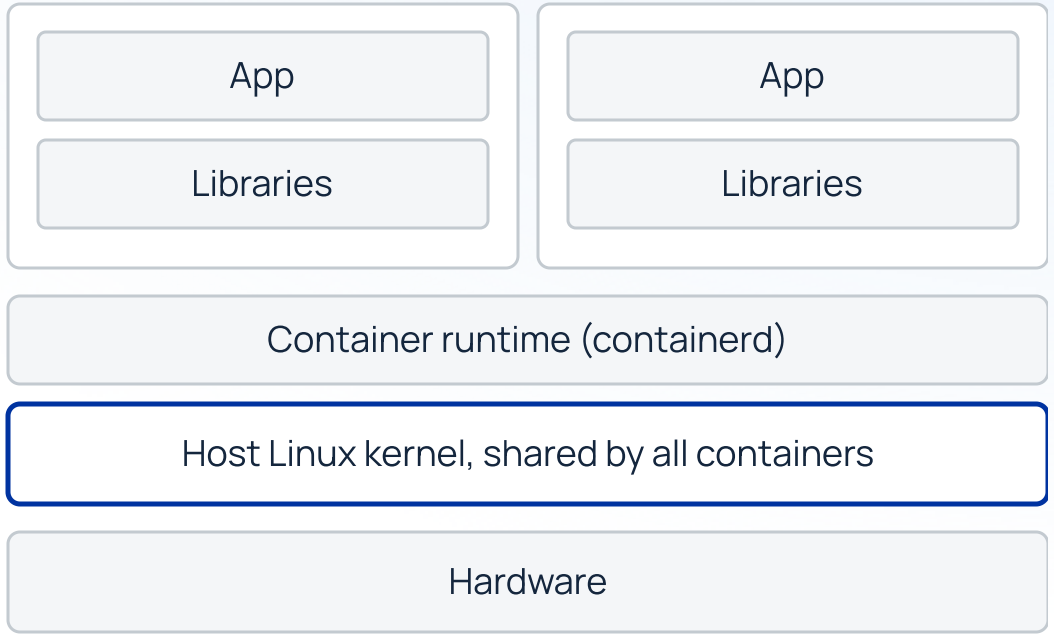
Images, layers, isolation, and registries

# Virtual machines and containers

### Virtual machines




### Containers




A virtual machine runs its own kernel on virtual hardware. A container is a group of ordinary processes on the host kernel, with an isolated view of the system.

Press ↓ for supporting detail: Containers vs. Virtual Machines

REFERENCE ONLY

DEEP DIVE / ISOLATION MODEL

# Containers vs. Virtual Machines

VIRTUAL MACHINES

## Hardware Virtualization

A hypervisor runs multiple complete operating systems on virtualized hardware. Each VM runs its own independent guest kernel, system daemons, and gigabytes of disk/memory, taking minutes to boot.

CONTAINERS

## Operating System Virtualization

Containers are ordinary Linux processes sharing the single host kernel. **Namespaces** partition what processes see (network, mounts, PIDs); **cgroups** restrict what resources they consume.

PERFORMANCE & PORTABILITY

Because containers share the host kernel, they start in milliseconds and consume megabytes rather than gigabytes. The exact same image runs identically on a developer laptop, CI server, or Kubernetes cluster.

# Linux namespaces isolate what a process can see

### PID Namespace
PROCESSES

Own process hierarchy. The container's primary process runs as PID 1, receiving signals like SIGTERM.

### Net Namespace
NETWORK

Private virtual interfaces, IP routing tables, firewall rules, and container-specific port allocations.

### Mount Namespace
FILESYSTEM

Isolated filesystem root constructed from container image layers, private from host disk paths.

### UTS Namespace
HOSTNAME

Independent system hostname and domain name, preventing conflicts across shared node hosts.

### IPC Namespace
INTER-PROCESS

Private POSIX/System V message queues, semaphores, and shared memory segments.

### User Namespace
PRIVILEGE

Maps container root (UID 0) to an unprivileged host UID, mitigating kernel breakout risks.

**Operating System Reality:** Containers are not lightweight virtual machines — they are standard host processes partitioned by Linux kernel namespaces.

# Control groups limit what a process can use

## CPU Enforcement

COMPRESSIBLE

- When a container reaches its limit, the Linux kernel throttles it.
- Process remains running; latency increases without crashing.
- Slow responses without restarts often indicate CPU throttling.

```
docker run --cpus 0.5 ...
```

## Memory Enforcement

INCOMPRESSIBLE

- When a container exceeds its limit, the kernel OOM killer terminates it.
- Process dies immediately with exit code 137 (`OOMKilled`).
- Memory leaks in applications trigger repeated container crash loops.

```
docker run --memory 256m ...
```

**Monitoring & Kubernetes Translation:** Inspect real-time cgroup counters with `docker stats`. Kubernetes requests and limits configure these exact same Linux cgroup settings.

# Images are stacks of read-only layers

| | |
|---|---|
| Container layer (read-write) | Created when the container starts. Removed with it. |
| Application code (COPY) | Rebuilt on every code change. |
| Dependencies (RUN pip install) | Rebuilt only when requirements.txt changes. |
| requirements.txt (COPY) | A few kilobytes. |
| Base image: python:3.12-slim | Shared with every image built on the same base. |

Each Dockerfile instruction that changes files adds a read-only layer. OverlayFS stacks the layers into one filesystem and adds a writable layer for each container.

# The OCI specifications

| Specification | What it defines | Examples |
|---|---|---|
| Image | The image manifest, configuration and layer format | Docker BuildKit, Buildah |
| Runtime | How to run a container from an unpacked filesystem and a configuration file | runc, crun |
| Distribution | The HTTP API for pushing and pulling images | Docker Hub, Amazon ECR, the k3d registry |

The Open Container Initiative (OCI) was founded in 2015 to standardise these formats. Because of them, an image built with Docker on your lab machine runs on containerd in k3d, on Amazon EKS and on Amazon ECS.

ECR: Elastic Container Registry.

# Tags and digests

```
k3d-registry.localhost:5000/flight-status:1.4.0
k3d-registry.localhost:5000/flight-status@sha256:9b1e5c0d…
```

- A tag is a name that points to an image. Anyone with push rights can point it at a different image.
- A digest is the SHA-256 (Secure Hash Algorithm, 256-bit) hash of the image manifest. It identifies one image and cannot change.
- `latest` is the default tag name. It does not mean the newest image.
- Deploy by digest, or by a tag that your pipeline never reuses.

# Container deployment artifacts (General DevOps)

REAL-WORLD SCENARIO

Netflix: the published Titus container-platform design.

- Titus was built to run containerized service and batch applications supporting Netflix's systems.
- A job specification names the container image, entry point and required resources.
- Course connection: an image is a deployment input. Record the exact artifact identity rather than relying on a mutable tag.

Source: Netflix, Titus overview.

# Container deployment artifacts (Retail)

SKIP REAL-WORLD SCENARIO

Shopify: its published Docker infrastructure journey.

- Shopify reported investing in Docker in 2014 to make builds and deployments predictable and repeatable.
- The stated benefits included simpler rollbacks and less configuration drift.
- Course connection: package the application in an image and replace a running version instead of patching it by hand.

Source: Shopify's Infrastructure Collaboration with Google.

# Dockerfile practices

### Minimal Base Image

ATTACK SURFACE

Start with `python:3.12-slim` over full OS images. Fewer preinstalled packages means faster pulls and a drastically smaller CVE footprint.

### Smart Layer Caching

BUILD SPEED

`COPY requirements.txt` and run `pip install` before copying source code. Prevents re-installing all dependencies on every commit.

### Least Privilege User

RUNTIME SECURITY

Explicitly run as non-root `USER 10001`. Prevents container breakout vulnerabilities from executing with root privileges on the host kernel.

### Build Context Hygiene

HYGIENE

Maintain strict `.dockerignore` rules excluding `.git`, test artifacts, virtual environments, and local `.env` credential files.

**Twelve-Factor Operational Rule:** Run a single main process per container and stream all application logs directly to `stdout` and `stderr`.

# A Dockerfile for a Python web service

```
FROM python:3.12-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY flightstatus/ ./flightstatus/
RUN useradd --uid 10001 --no-create-home app
USER 10001
EXPOSE 8000
CMD ["uvicorn", "flightstatus.main:app", "--host", "0.0.0.0", "--port", "8000"]
```

- Dependencies are installed before the code is copied, so a code change reuses the cached dependency layer.
- The server runs as user 10001, which has no special rights on the host.

# Container risks in NIST SP 800-190

| Area | Example risk | What this course does about it |
|---|---|---|
| Image | Known vulnerabilities in packages; secrets stored in a layer | Trivy scan in the pipeline; secret scanning on push |
| Registry | Stale or untrusted images; unencrypted connections | Deploy by unique tag or digest; TLS for real registries such as ECR |
| Orchestrator | Too much access for people and pipelines | A namespaced ServiceAccount with RBAC |
| Container | Running as root; no resource limits | Non-root user; requests and limits |
| Host OS | One kernel shared by all containers | Patched hosts, or no hosts to manage with Fargate |

NIST SP 800-190 groups container risks into these five areas and gives countermeasures for each (Souppaya et al., 2017).

NIST: National Institute of Standards and Technology. SP: Special Publication. TLS: Transport Layer Security. RBAC: role-based access control.

# Software bills of materials and vulnerability scanning

TIER 1 · CATALOG

### The Ingredient List (SBOM)

A machine-readable catalog (**CycloneDX** or **SPDX**) documenting every OS package and language dependency version in the image without rebuilding.

TIER 2 · SCAN

### Automated CVE Matching

Scanners like **Trivy** cross-reference SBOM packages against CVE databases (NVD, GitHub Advisories) to identify vulnerabilities with assigned severities.

TIER 3 · GATE

### Actionable Policy Enforcement

Enforce pipeline gates on actionable findings: fail CI builds on `CRITICAL` issues with available upstream patches (using `--ignore-unfixed`).

**The Continuous Vulnerability Clock:** An image with zero CVEs today will develop CVEs tomorrow as new exploits are published — continuous re-scanning of deployed images is mandatory.

# Signing and provenance

- A signature shows who published an image and that it has not changed since.
- Sigstore's cosign signs an image digest and stores the signature in the registry next to the image.
- Provenance records how an image was built, including the source commit, the build system and the build steps.
- Supply-chain Levels for Software Artifacts (SLSA) Build levels 1 to 3 ask for provenance, then provenance signed by a hosted build platform, then a hardened build platform.
- A cluster admission policy can refuse images that are not signed by a trusted key.

# Docker Compose and AWS Fargate

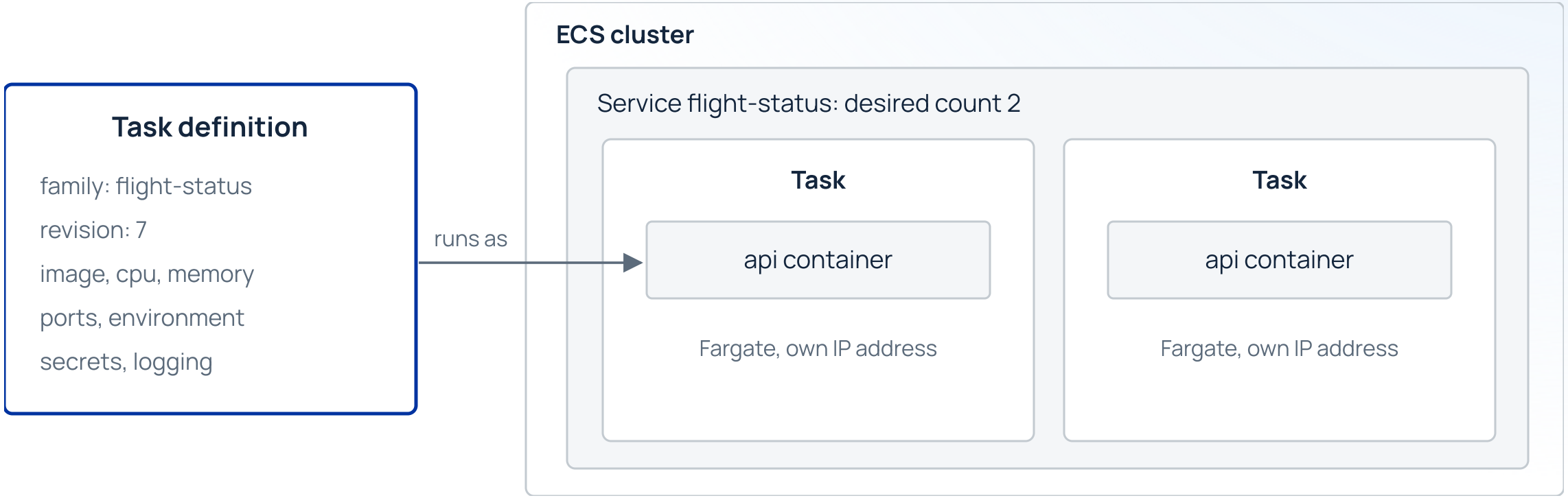
Multi-container orchestration and serverless containers

# Docker Compose describes a multi-container application

```yaml
services:
  app:
    build: .
    ports:
      - "8000:8000"
    environment:
      DATABASE_URL: postgresql://flights:flights@db:5432/flights
    depends_on:
      - db
  db:
    image: postgres:17-alpine
    environment: {POSTGRES_USER: flights, POSTGRES_PASSWORD: flights, POSTGRES_DB: flights}
```

Compose creates one network for the project, and each service is reachable by its name. The API reaches the database at the host name `db`.

# Amazon ECS concepts




A task definition is a versioned JSON document. A task is one running copy of it, and a service keeps the desired number of tasks running.

# Fargate runs tasks without servers to manage

| | Elastic Compute Cloud (EC2) launch type | Fargate |
|---|---|---|
| Who manages the hosts | You, as a group of EC2 instances | AWS |
| What you size | Instances, then the tasks placed on them | The CPU and memory of each task |
| Network mode | bridge, host or awsvpc | awsvpc only, so each task gets its own network interface |
| Billing | Per instance | Per task, for vCPU (virtual CPU) and memory, per second |
| Access to the host | Possible | None |

At United, containers run on Amazon ECS with Fargate.

# Mapping Compose to an ECS task definition

| compose.yaml | Task definition (JSON) |
|---|---|
| `image:` | `containerDefinitions[].image` |
| `ports: ["8000:8000"]` | `portMappings: [{"containerPort": 8000}]` |
| `environment:` | `environment: [{"name": ..., "value": ...}]` |
| A password in `environment:` | `secrets: [{"name": ..., "valueFrom": "<Secrets Manager ARN>"}]` |
| `healthcheck:` | `healthCheck.command` |
| `deploy.resources.limits` | Task-level `cpu` and `memory` |
| Logs on standard output | `logConfiguration` with the `awslogs` driver |
| `depends_on:` | `dependsOn` inside one task; separate services otherwise |

# What changes when the task runs in AWS

- Each Fargate task gets its own network interface and private IP address in a virtual private cloud (VPC) subnet.
- The task execution role lets ECS pull the image and read secrets. The task role is what the application uses to call AWS APIs.
- An Application Load Balancer sends requests to healthy tasks. It replaces the port mapping on your machine.
- The database is a managed service such as Amazon Relational Database Service (RDS).
- Logs go to Amazon CloudWatch Logs.

HANDS-ON LAB

# Lab 07: Build, Scan and Describe a Container

Day 3 · 35–45 minutes

HANDS-ON LAB

# Lab 07: Build, scan and describe a container

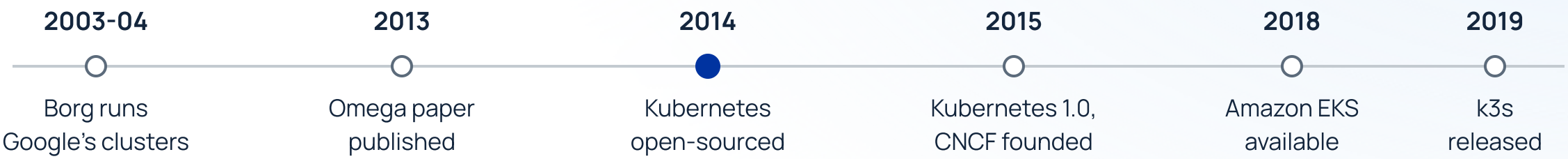
Open `labs/07-containers.md` in the course repository.

- Write a Dockerfile, build the flight-status image and inspect its layers.
- Push the image to the local registry.
- Install Trivy, scan the image and produce an SBOM.
- Run the API with Postgres in Docker Compose, then map the Compose file to an ECS task definition.

# Kubernetes Core Concepts

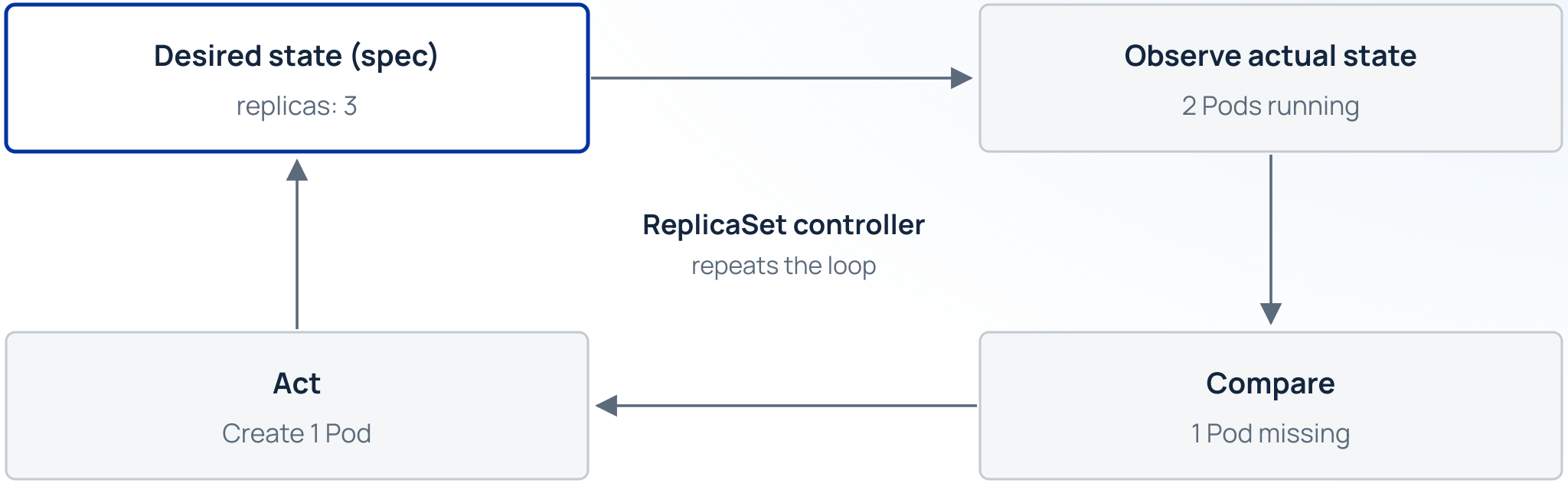
Control plane architecture, Pods, Deployments, and Services

# From Borg to Kubernetes



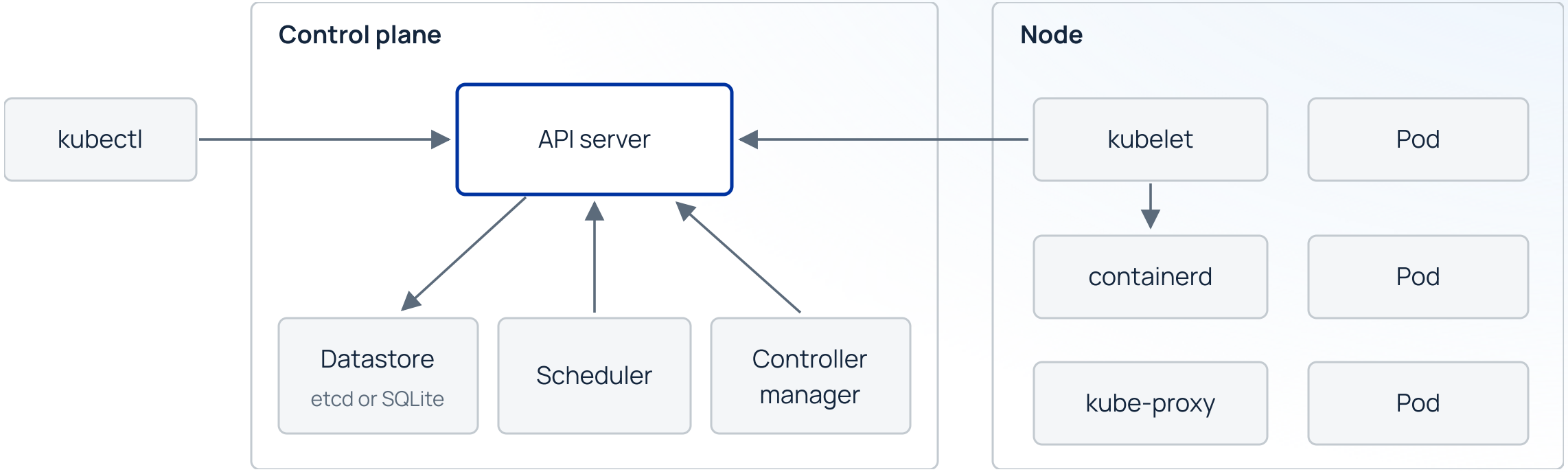

Burns et al. (2016) describe what Google learned from Borg and Omega. Kubernetes adopted labels instead of numbered jobs, one IP address per Pod, and independent controllers that reconcile state.

# The declarative API and reconciliation loops



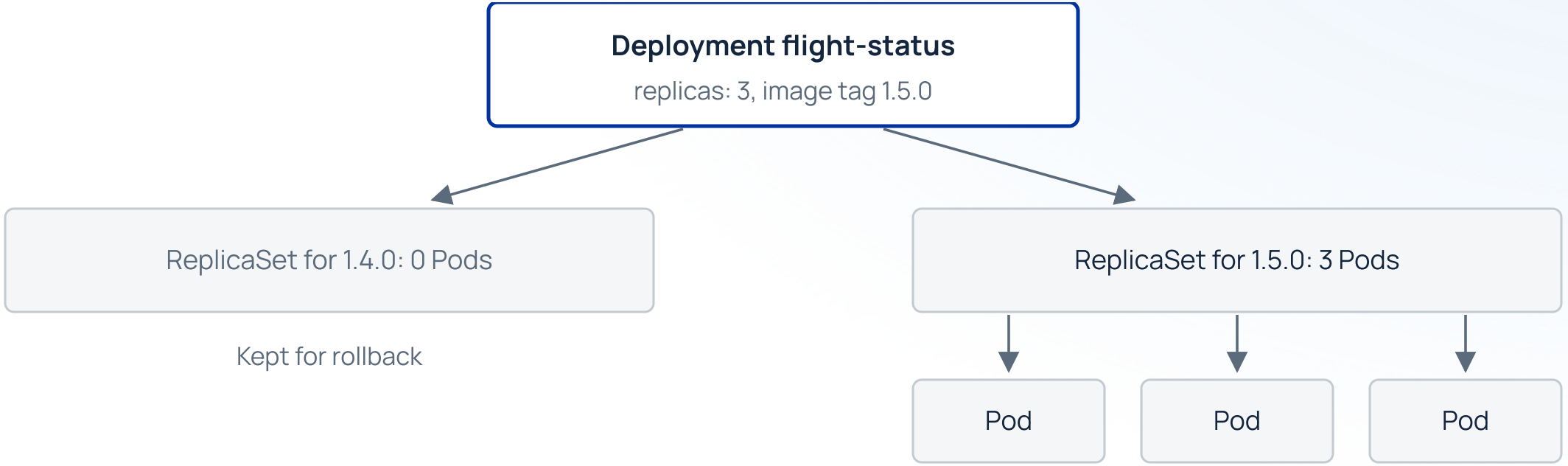

Every Kubernetes controller runs this loop. A deleted Pod comes back because the ReplicaSet controller sees two Pods where the spec asks for three, and creates one.

# The control plane and the nodes




Every component talks to the API server, and only the API server uses the datastore. In k3d, one container (`k3d-devops-server-0`) runs the control plane and your Pods.

# Deployments, ReplicaSets and Pods




A Pod is one or more containers that share an IP address and volumes. A Deployment creates a new ReplicaSet for each new Pod template, scales the old one to zero and keeps it for rollback.

Press ↓ for supporting detail: Hierarchy of Kubernetes Objects

REFERENCE ONLY



# Hierarchy of Kubernetes Objects

1. POD

## Atomic Unit

The smallest schedulable unit. One or more co-located containers sharing an IP address, storage volumes, and IPC/localhost communication.

2. REPLICASET

## Replica Governor

Guarantees that a specified count of identical Pods remain running. If a worker node crashes or a Pod terminates, it launches a replacement immediately.

3. DEPLOYMENT

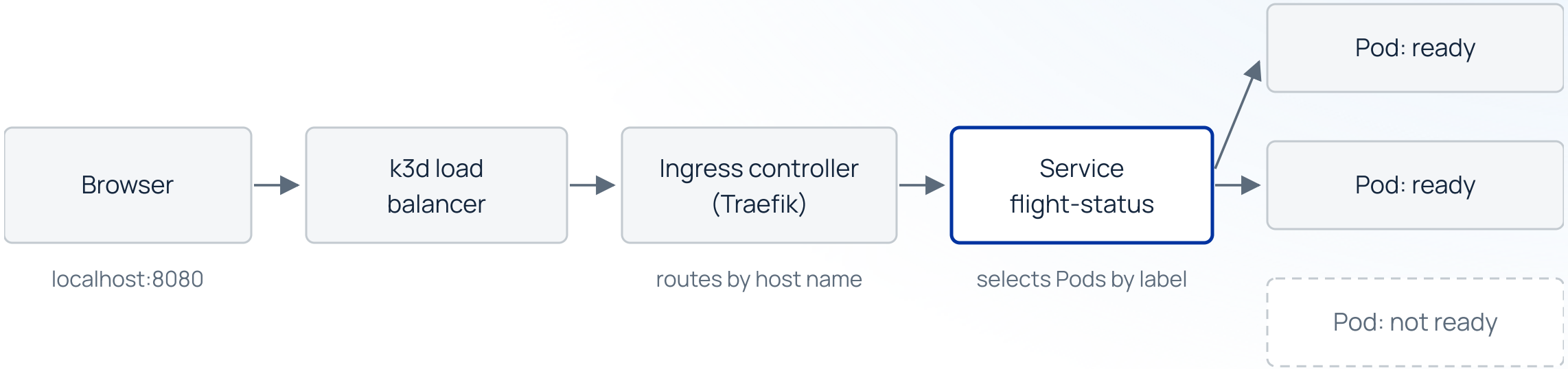
## Rollout Controller

Higher-level controller managing ReplicaSets. Drives zero-downtime rolling updates, tracks revisions, and powers instant rollbacks (`kubectl rollout undo`).

**DECLARATIVE OBJECT HIERARCHY**

Engineers define Deployments in Git YAML files and rarely manage ReplicaSets or Pods directly. The Deployment controller automates version upgrades and replica health under the hood.

# Services and Ingress




A Service gives a set of Pods one stable name and address, and sends traffic only to Pods that pass their readiness probe. An Ingress routes HTTP requests to Services by host name and path.

Press ↓ for supporting detail: Why do we need a Service?

REFERENCE ONLY

DEEP DIVE / KUBERNETES NETWORKING

# Why Do We Need a Kubernetes Service?

THE PROBLEM

## Ephemeral Pod IPs

Pods are mortal. Every restart, reschedule, or autoscaling event assigns a new dynamic IP address. Clients cannot rely on direct Pod IP endpoints.

THE SOLUTION

## Stable Virtual IP & DNS

A Service provides a permanent ClusterIP and internal DNS record (`flight-status.flightops-dev.svc.cluster.local`) that never changes across deployments.

TRAFFIC DISTRIBUTION

## Label Selector & Probes

The Service tracks Pods with matching labels (e.g. `app: flight-status`). Traffic is routed only to Pods currently passing readiness health probes.

SERVICE VS. INGRESS

A **Service** manages internal Layer 4 (TCP/UDP) routing and load balancing within the cluster. An **Ingress** handles external Layer 7 (HTTP/HTTPS) routing, host matching, and SSL/TLS termination.

# Liveness, readiness and startup probes

STARTUP PROBE

### Initialization Guard

*"Has the application finished starting up?"*

Pauses liveness and readiness checks until passing. If it fails past threshold, kubelet restarts the container. Protects slow-starting services from premature termination.

LIVENESS PROBE

### Deadlock Recovery

*"Is the process deadlocked, crashed, or hung?"*

Kubelet terminates and restarts the container immediately. Keep checks strictly internal (`/health`) — never check external databases here!

READINESS PROBE

### Traffic Gatekeeper

*"Can this Pod safely accept incoming traffic right now?"*

Removes Pod from Service endpoints without restarting. Enables zero-downtime rolling updates (`/ready` tests internal readiness and database connectivity).

**Core Architectural Rule:** Liveness restarts the process; Readiness controls traffic routing.

# Requests, limits and quality of service

### Requests · Scheduling Floor

Minimum CPU and memory reserved for the container. `kube-scheduler` sums requests to place Pods only on nodes with verified unreserved capacity.

### Limits · Runtime Ceiling

Hard maximum capacity enforced via Linux cgroups. CPU usage is throttled; memory spikes beyond limits trigger immediate kernel `OOMKilled` termination.

### Guaranteed — TOP PRIORITY

`requests == limits` for all containers. Highest stability; evicted last under node memory pressure.

### Burstable — STANDARD

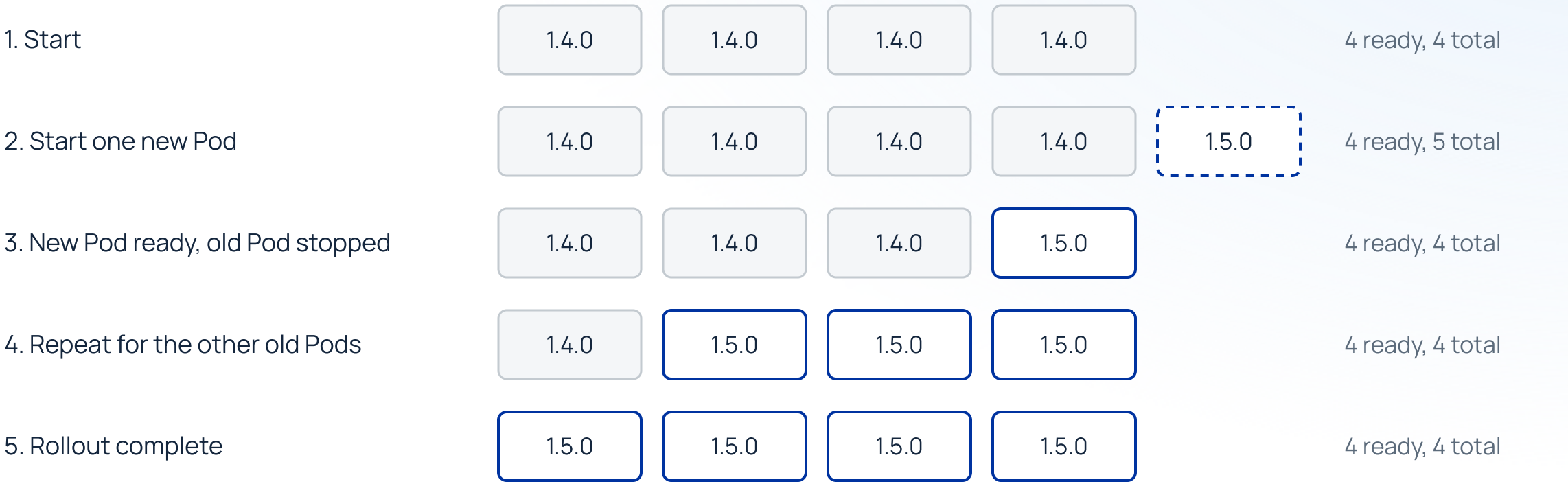
`requests < limits`. Allowed to burst above floor; evicted based on memory over-subscription.

### BestEffort — LOWEST

No requests or limits specified. High eviction risk; first to be terminated when node memory runs low.

**Resource Sizing Discipline:** Always specify both requests and limits — prevents noisy-neighbor CPU starvation and satisfies cluster ResourceQuotas.

# A rolling update, step by step




Settings: 4 replicas, maxSurge 1, maxUnavailable 0. There are never fewer than 4 ready Pods or more than 5 Pods. The dashed Pod has started but is not ready yet.

# Kubernetes operations (General DevOps)

REAL-WORLD SCENARIO

Spotify: migration from its homegrown Helios orchestrator.

- The published case study describes adopting Kubernetes while continuing to run Helios during migration.
- Running them alongside each other let teams validate Kubernetes before moving all workloads.
- Course connection: preserve a working path while checking a replacement under realistic conditions.

Source: Kubernetes, Spotify case study.

# Kubernetes operations (Retail)

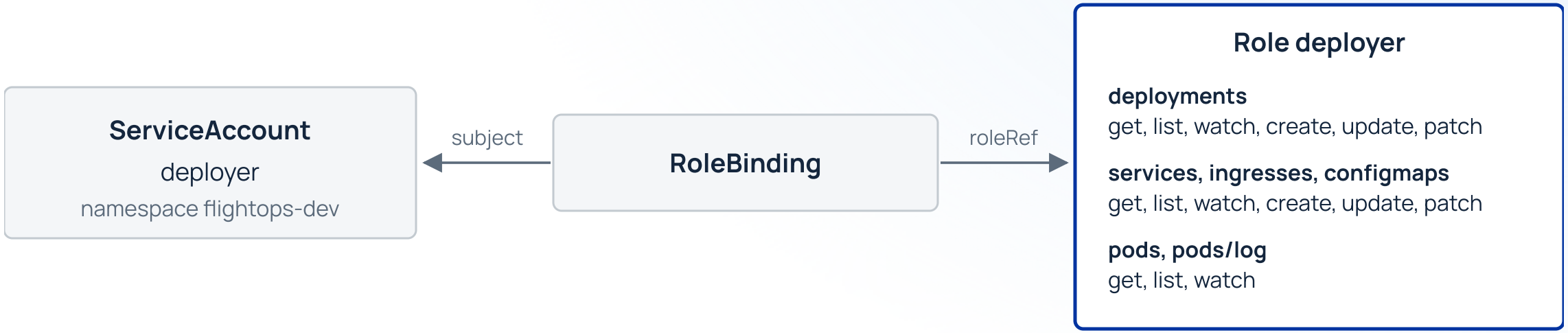
SKIP REAL-WORLD SCENARIO

Shopify: rollout feedback with krane and Shipit.

- Shopify's krane deploy tool uses kubectl apply and watches requested changes until they roll out.
- It reports actionable pass/fail results and diagnostic information when deployment fails.
- Course connection: submitting manifests is only the start. Inspect rollout health before calling a deployment successful.

Source: Shopify, krane project README.

# Role-based access control




A Role allows verbs on resource types in one namespace, and a RoleBinding grants the Role to a user, group or ServiceAccount. RBAC rules only add permissions, so anything a Role does not list is refused.

# k3d and Amazon EKS

| | k3d on your lab machine | Amazon EKS |
|---|---|---|
| Control plane | A k3s server in one Docker container | Run by AWS across availability zones |
| Nodes | The same container | EC2 managed node groups or Fargate |
| Ingress | Traefik, bundled with k3s | AWS Load Balancer Controller with an Application Load Balancer (ALB) |
| Registry | `k3d-registry.localhost:5000` over HTTP | Amazon ECR over HTTPS with Identity and Access Management (IAM) |
| Workload identity | ServiceAccount tokens | ServiceAccount tokens plus IAM roles (EKS Pod Identity) |

The Kubernetes API, kubectl, the manifests and RBAC are the same in both. That is why the Lab 08 manifests would run on EKS with only the image and Ingress settings changed.

KNOWLEDGE CHECK

# Section Quiz 1: Kubernetes

Twelve questions. Five minutes, including feedback.

- Use the Kahoot game PIN shared by your instructor.
- After each answer, compare the reasoning with your own.

HANDS-ON LAB

# Lab 08: Kubernetes Deployments, Rollouts and Rollbacks

Day 3 · 40–50 minutes

HANDS-ON LAB

# Lab 08: Kubernetes deployments, rollouts and rollbacks

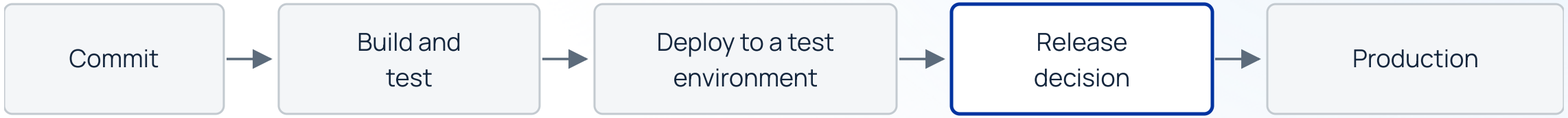
Open `labs/08-kubernetes-rollouts.md` in the course repository.

- Deploy the API with a Deployment, Service and Ingress, including probes, requests and limits.
- Scale it, delete a Pod and watch Kubernetes replace it.
- Roll out a new version, then a broken image and a version that fails its readiness probe.
- Roll back and read the rollout history.

# Continuous Delivery

Deployment strategies, verification gates, and automated rollbacks

# Continuous delivery and continuous deployment




Delivery: a person decides.
Deployment: the pipeline decides.

Continuous delivery keeps every change that passes the pipeline ready to release at any time (Humble and Farley, 2010). Continuous deployment releases each of those changes automatically.

Press ↓ for supporting detail: Delivery vs. Deployment

REFERENCE ONLY

DEEP DIVE / RELEASE STRATEGY

# Delivery vs. Deployment Mental Model

CONTINUOUS DELIVERY (CD)

## Releasable on Demand

Every commit that passes automated builds and tests is deployed to staging. The codebase is continually proven releasable, but production rollout pauses at an explicit business approval gate.

CONTINUOUS DEPLOYMENT (CD)

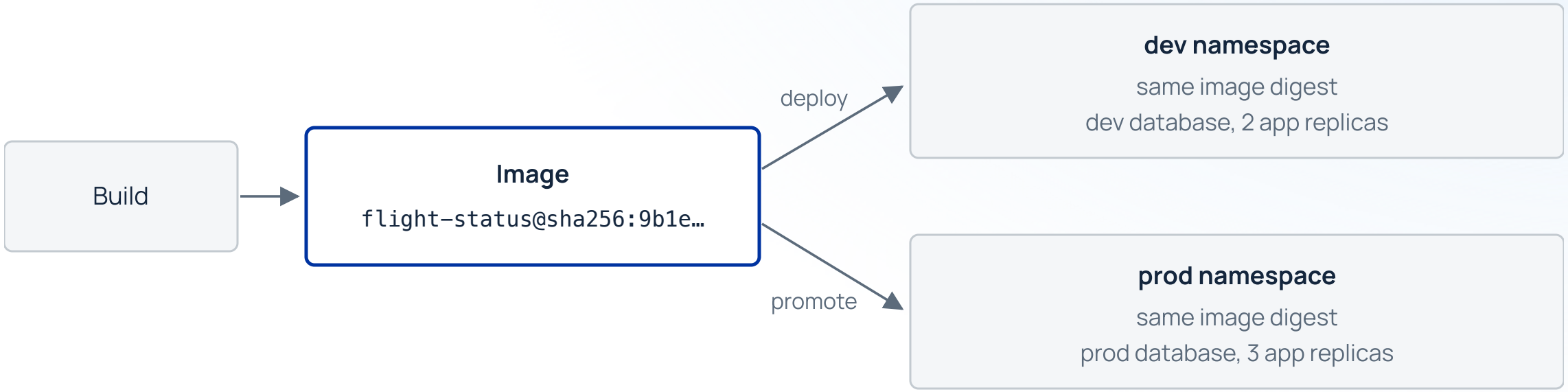
## Direct to Production

Every commit passing automated pipeline gates is deployed directly to production with zero manual intervention. Relies on automated canary verification and automated rollbacks.

ENTERPRISE REALITY

Most enterprise and regulated organizations practice **Continuous Delivery**: pipelines automate all verification and staging deployments, while business owners retain control over production release timing.

# Build once, promote the same artifact



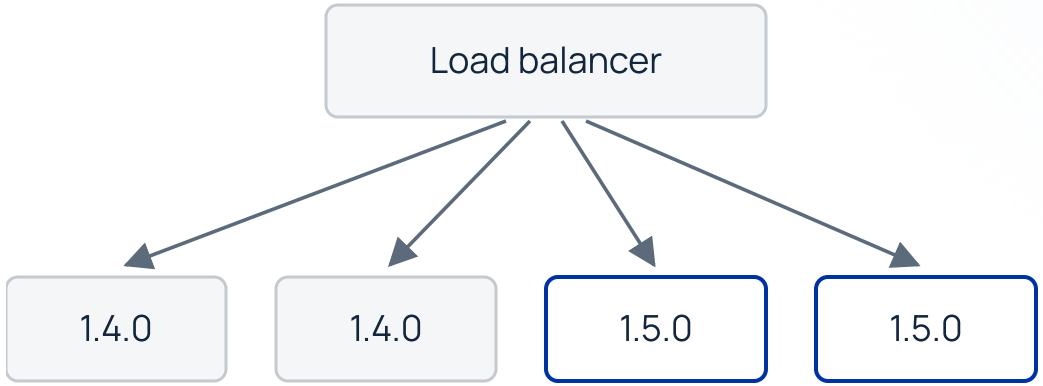

The image that passed the checks in dev is the image that runs in prod. Settings that differ between environments come from the environment, as variables and Secrets (Wiggins, 2011).

# Approvals

- An approval is a pipeline step that waits for a named person or group to confirm.
- Use one where a person adds information the pipeline does not have, such as a change window or a business sign-off.
- Every approval adds waiting time to the lead time for changes.
- The Harness platform has Harness, Jira and ServiceNow approval steps, with timeouts.
- Harness Open Source has no approval step. In Lab 09, a person promotes a release to prod by pushing a git tag.

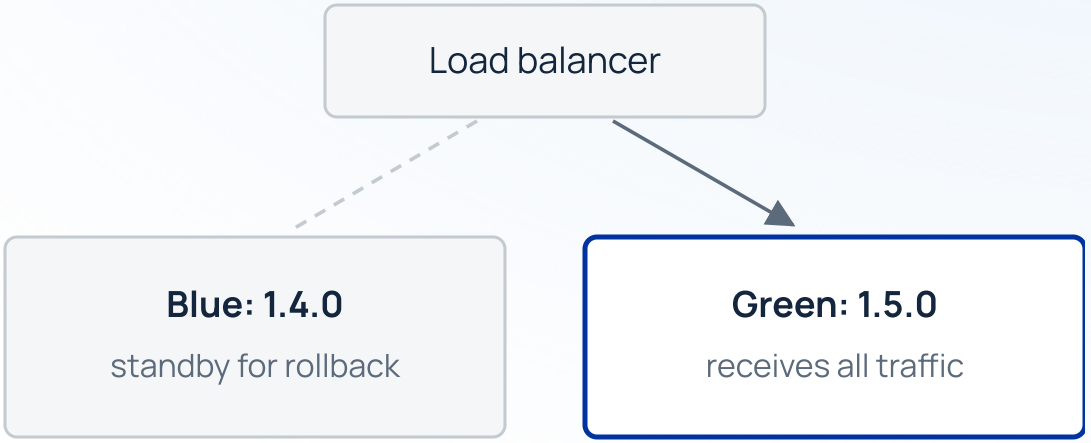
# Rolling and blue-green deployments

### Rolling




Old and new versions serve traffic during the rollout.

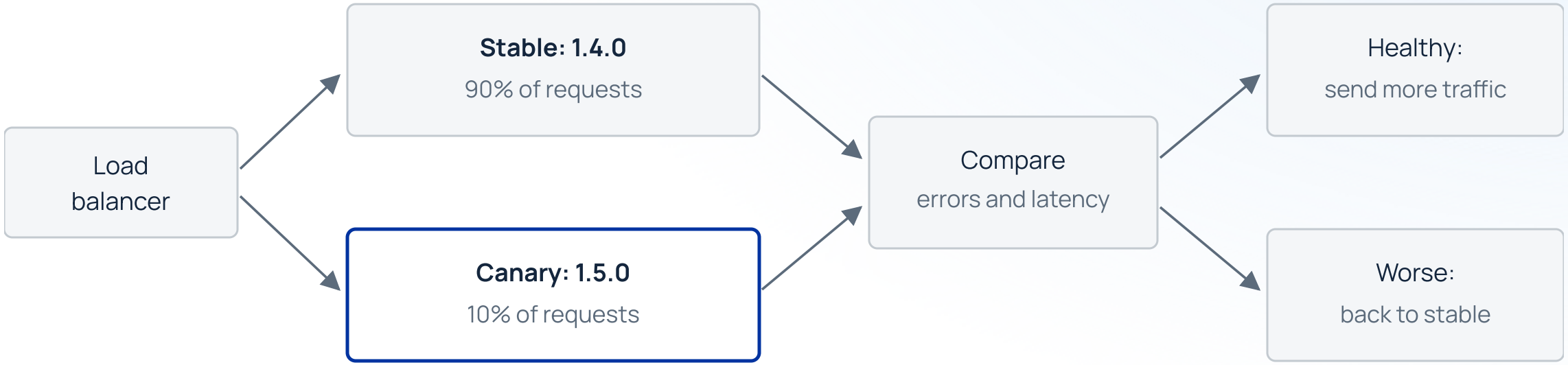
### Blue-green




Traffic moves to the new environment in one switch.

A rolling deployment needs little extra capacity but runs two versions side by side for a while. Blue-green needs two full environments and rolls back by switching traffic back (Fowler, 2010).

# Canary releases



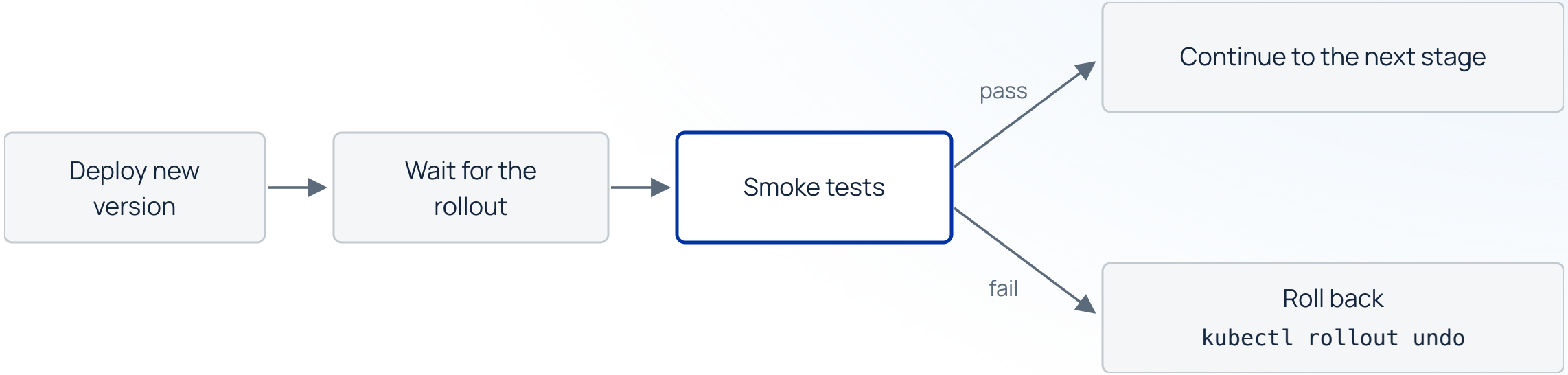

A canary release sends a small share of real traffic to the new version and compares it with the current version before sending more (Sato, 2014). With plain Kubernetes, the split follows the Pod count.

# Feature flags separate deployment from release

```
if flags.is_enabled("new-departures-board", user):
    return new_departures_board(flights)
return departures_board(flights)
```

- The new code is deployed to production, switched off.
- Turning the flag on releases the feature to some users or to everyone, without a deployment.
- Turning it off again is the fastest rollback for that feature.
- Each flag needs an owner and a removal date. Old flags multiply the code paths you must test (Hodgson, 2017).

# Automated verification and rollback




A smoke test calls the new version the way a user would: a health check, a version check, one real request. The Lab 09 smoke test checks the version through the ingress; a "Going further" task adds the other two. A step with `when: failure()` rolls back when any earlier step fails.

# Lab 09 steps and the Harness platform

| Lab 09 step in Harness Open Source | Harness platform feature for the same job |
| --- | --- |
| Run step with pytest and a Postgres background step | Build stage with a Run step and a Background step |
| Docker plugin step that builds and pushes the image | Build and Push to Docker step with a registry connector |
| Trivy scan step | Security Testing Orchestration with the Aqua Trivy step |
| kubectl step with a kubeconfig secret | Deploy stage: service, environment, infrastructure definition, Kubernetes connector and delegate |
| Smoke test step | Verify step (Continuous Verification) or a Shell Script step |
| Rollback step with `when: failure()` | Failure strategy with the Rollback Stage action |
| Promote pipeline started by a release tag | Approval stage, then a Deploy stage for the prod environment |

# DORA metrics revisited

DORA: DevOps Research and Assessment.

| Metric | Practices from this course that improve it |
| --- | --- |
| Deployment frequency | Small pull requests, an automated pipeline, one image promoted through environments |
| Lead time for changes | A pipeline on every push, fast tests, few manual approvals |
| Change failure rate | Tests and quality gates, image scanning, readiness probes, canary releases |
| Failed deployment recovery time | Immutable images, kept ReplicaSets, automated rollback |

Forsgren et al. (2018) found that teams with high delivery speed also had high stability. The same practices improved both.

KNOWLEDGE CHECK

# Section Quiz 2: Continuous Delivery

Twelve questions. Five minutes, including feedback.

- Use the Kahoot game PIN shared by your instructor.
- After each answer, compare the reasoning with your own.

HANDS-ON LAB

# Lab 09: End-to-End Delivery Pipeline

Day 3 · 60–75 minutes

HANDS-ON LAB

# Lab 09: End-to-end delivery pipeline

Open `labs/09-end-to-end-delivery.md` in the course repository.

- Build one pipeline: test, build, scan, push, deploy to dev and smoke test.
- Promote the same image to prod by pushing a release tag.
- Release a broken version on purpose and watch the pipeline roll it back.
- Compare each step with its Harness platform feature.

RECAP

# Day 3 summary

MODULE 1

## Containers & Images

- Multi-stage Docker builds for minimal attack surface
- Non-root user execution & layer caching
- OCI specifications: tags, digests & immutability
- Compose definitions to ECS/Fargate tasks

MODULE 2

## Supply Chain Security

- Automated vulnerability scanning with Trivy
- Software Bill of Materials (CycloneDX SBOM)
- NIST SP 800-190 container security risks
- Shift-left security quality gates in CI/CD

MODULE 3

## Kubernetes Orchestration

- Declarative Deployments, ReplicaSets & Pods
- Cluster networking via Services & Ingress
- Liveness, readiness & startup health probes
- Zero-downtime rolling updates & rollback safety

MODULE 4

## Continuous Delivery

- Build once: promote identical artifact across environments
- Deployment vs. delivery: feature flags & approvals
- Progressive strategies: rolling, blue-green & canary
- Automated verification & instant rollback on failure

COURSE MILESTONE **Complete CI/CD Pipeline: Automated, Secure & Production-Ready Delivery**

# What the three days covered

DAY 1 · LABS 01-03

## Continuous Integration

- Trunk-based Git development & small pull requests
- Automated unit, integration & API test pyramid
- Shift-left static analysis & linting with Ruff
- Harness CI pipeline validation on every change

DAY 2 · LABS 04-06

## Infrastructure as Code

- Declarative Terraform & dependency graphs
- Remote state backends & distributed locking
- PR plan / merge apply automated delivery engine
- Secret-zero elimination via OIDC workload identity

DAY 3 · LABS 07-09

## Containers & Delivery

- Optimized multi-stage Docker packaging & non-root
- Vulnerability scanning with Trivy & CycloneDX SBOMs
- Kubernetes Deployments, Services, and Ingress
- Zero-downtime rolling updates & automated rollback

CAPSTONE OUTCOME **Production-Ready Delivery:** The `flight-status` API is continuously tested, built, scanned, and safely deployed to declarative cloud environments.

CHAT WATERFALL

# Put it to work

Complete this privately:

When [specific delivery situation] occurs, I will [one practice], using [artifact or evidence]. I will know it worked when [observable result].

- Choose a change you can discuss safely, or use the course's synthetic service.
- Check that your plan names a situation, an action, evidence and a success signal.
- Sharing in chat or aloud is optional; keep production details private.

KEY POINTS

# Next steps and feedback

Example: when our next service update is ready, I will add a version smoke test. Its response must match the expected commit before we promote the image.

- Keep the course repository. Every lab runs again on a new Ubuntu 24.04 machine.
- Open one of your team's pipelines in the Harness platform and find each stage from the Lab 09 mapping table.
- Pluralsight has paths on Docker, Kubernetes, Terraform and AWS for more depth on each topic.
- Please fill in the course survey. The next version of this course uses your answers.

# References


- Burns, B., Grant, B., Oppenheimer, D., Brewer, E., and Wilkes, J. (2016). Borg, Omega, and Kubernetes. ACM Queue 14(1).
- Forsgren, N., Humble, J., and Kim, G. (2018). Accelerate. IT Revolution.
- Fowler, M. (2010). BlueGreenDeployment. martinfowler.com
- Hodgson, P. (2017). Feature Toggles (aka Feature Flags). martinfowler.com
- Humble, J., and Farley, D. (2010). Continuous Delivery. Addison-Wesley.
- Sato, D. (2014). CanaryRelease. martinfowler.com
- Souppaya, M., Morello, J., and Scarfone, K. (2017). Application Container Security Guide. NIST SP 800-190.
- Wiggins, A. (2011). The Twelve-Factor App. 12factor.net
- Open Container Initiative. Image Format, Runtime and Distribution Specifications. opencontainers.org
- SLSA (2023). Supply-chain Levels for Software Artifacts, v1.0. slsa.dev
- Linux manual pages: namespaces(7), cgroups(7). man7.org
- Kubernetes documentation: Deployments, probes, resource management, RBAC. kubernetes.io/docs
- Amazon Web Services. Amazon ECS Developer Guide: task definitions and Fargate. docs.aws.amazon.com
- DORA (2024). Accelerate State of DevOps Report. dora.dev
- Harness documentation: CI, CD, STO (Security Testing Orchestration) and failure strategies. developer.harness.io

# Let’s Connect

Scan the QR code or click the link below to stay in touch



**Kangeyan Passoubady**

linkedin.com/in/kpassoubady

Point your phone camera at the QR code

# Thank You

Intermediate DevOps · Day 3: Containers, Kubernetes and Continuous Delivery



KavinSchool